

IN422 - TP3 - Arbres binaires, arbres binaires de recherche, arbres implicites et tas.

Grenoble INP – Esisar – année 2025-26



Version du 1^{er} décembre 2025



Objectif pédagogique

- Manipuler différents types de structures arborescentes.
- Implémenter des structures de données en C en utilisant un ramasse-miettes.
- Illustrer l'usage de telles structures via des applications d'exemple.



Evaluation

Chaque binôme rendra:

- une archive contenant le code réalisé;
- séparément de cette archive, un compte-rendu au format .pdf commentant les choix qui ont été faits, et contenant les analyses de vos expérimentations.

Il vous est demandé de produire vous-mêmes le code demandé, ainsi que le compte-rendu. Les emprunts aux collègues, et la délégation du travail à des IAs, ne vous permettront pas de récolter les bienfaits attendus: renforcer des savoirs, développer des savoir-faire. Dans le premier cas, il s'agit de plagiat, qui peut vous exposer à des sanctions.

Pour chaque code demandé, considérer qu'ils vous est également demandé d'écrire des tests illustrant le bon fonctionnement de ce code. Ce ne sera jamais précisé dans les questions de l'énoncé, mais c'est à faire systématiquement.

Vous prendrez soin d'inclure vos deux patronymes dans chacun des deux fichiers. Les fichiers sont à déposer sur Chamilo, au plus 7 jours après la séance de TP.



Etape 0 : quelques explications

Dans tout ce TP, nous manipulerons des structures de données en laissant la gestion mémoire aux bons soins d'un gestionnaire automatique de mémoire fonctionnant selon le principe du *ramasse-miettes*, dans le même esprit que le TP précédent sur les listes.

L'ensemble du TP peut être compris comme la suite du TP précédent. Dans le TP précédent, vous avez manipulé des listes chaînées. Dans ce TP, nous partons à la découverte de différentes structures arborescentes, c'est-à-dire en forme d'arbres.

Après avoir désarchivé le code fourni, copiez les fichiers `fonctions_listes.c` et `fonctions_listes.h` de votre précédent TP dans le répertoire courant.



Etape 1 : prise en main des arbres binaires.

Nous allons commencer par implémenter quelques fonctions utilitaires sur un type d'arbres embarquant ses valeurs dans ses nœuds internes. Ce type n'aura pas d'application directe; son usage sera illustré à l'étape suivante, où il servira de structure de base pour l'implémentation d'une structure de donnée plus riche, les *arbres binaires de recherche*.

Commençons par décrire ce type de données.

Un *arbre binaire sans valeur aux feuilles* est soit l'arbre vide, soit une cellule embarquant une valeur et deux arbres. Par convention, et pour les distinguer, on nomme ces arbres "l'arbre gauche" et "l'arbre droit".

Les arbres binaires sans valeur aux feuilles sont donc un type abstrait de données répondant à l'interface suivante:

```
arbre arbre_vide()
bool est_arbre_vide(arbre a)
arbre arbre_constructeur(t_val v, arbre g, arbre d)
t_val arbre_valeur(arbre a)
arbre arbre_gauche(arbre a)
arbre arbre_droit(arbre a)
```

avec les équations:

- `est_arbre_vide(arbre_vide()) = Vrai`
- `est_arbre_vide(arbre_constructeur(v, g, d)) = Faux`
- `arbre_valeur(arbre_constructeur(v, g, d)) = v`
- `arbre_gauche(arbre_constructeur(v, g, d)) = g`
- `arbre_droit(arbre_constructeur(v, g, d)) = d`

Notez la similarité avec les listes chaînées; les deux différences sont la présence d'un membre de plus; et le renommage des fonctions, pour se rapprocher des nomenclatures usuellement employées pour les arbres.

Vous êtes invités à lire le module `arbre.h/arbre.c`, qui propose une implémentation de ce type abstrait de données.

Pour la mise au point de votre code, vous pouvez utiliser la fonction `arbre_affiche()`, qui permet de visualiser un arbre binaire.

- Implémentez la fonction `arbre_profondeur()` du module `fonctions_arbres.c`. La profondeur de l'arbre vide est 0.
- Implémentez la fonction `arbre_nombre_elements()` du module `fonctions_arbres.c`
- Implémentez la fonction `arbre_elements()` du module `fonctions_arbres.c`. Pour cette fonction, il vous est demandé de ranger dans une liste, dans cet ordre, les valeurs du sous-arbre gauche, puis la valeur du noeud considéré, puis les valeurs du sous-arbre droit. Cet ordre de traitement est appelé *ordre infixe*.
- Implémentez la fonction `arbre_source_c()` du module `fonctions_arbres.c`: elle doit afficher à l'écran une expression en langage C construisant l'arbre passé en paramètre.



Etape 2 : arbres binaires de recherche

Un arbre binaire de recherche est un arbre binaire ayant la propriété que toutes les étiquettes stockées dans le sous-arbre gauche d'un noeud sont plus petites que l'étiquette de ce noeud, et que toutes les étiquettes stockées dans le sous-arbre droit d'un noeud sont plus grandes que l'étiquette de ce noeud.

Les arbres binaires de recherche sont des collections: leur interface comprend des membres pour la recherche, l'ajout et le retrait, comme vous pouvez le voir dans `abr.h`.

Du point de vue de l'implémentation, les arbres binaires de recherche sont implémentés comme de simples arbres binaires. Le maintien de la propriété d'être un arbre binaire de recherche sera garantie pour l'utilisateur manipulant des arbres binaires de recherche en utilisant seulement l'interface du fichier `abr.h`.

Les utilitaires de `fonctions_arbres.h` peuvent toutefois être utilisés pour l'inspection, et pour d'autres usages ne visant pas à créer de nouveaux arbres binaires de recherche.

- Implémentez la fonction `abr_cherche(abr *a, int n)`, qui cherche l'élément `n` dans l'arbre binaire de recherche `a`. Un arbre vide ne contient pas l'élément `n`. Dans le cas de l'arbre non vide, comparez `n` avec la valeur à la racine de l'arbre. Suivant le résultat de la comparaison (plus petit, plus grand, égal), relancez la recherche dans le sous-arbre gauche, ou droit; ou répondez simplement que l'élément est présent.
- Implémentez la fonction `abr_insere(abr *a, int n)`, qui rajoute l'élément `n` à l'arbre `a` (on ajoute `n`, qu'il se trouve déjà dans l'arbre ou pas.) Pour cela, distinguez le cas de l'arbre vide. Pour un arbre non-vide, comparez `n` à la valeur à la racine, et relancez récursivement l'ajout dans un sous-arbre ou dans l'autre, en fonction de cette comparaison.
- Ecrire une fonction `abr_max(abr *a)`, qui calcule la valeur maximum présente dans l'arbre `a`. Si `a` est vide, générez une erreur, et arrêtez le programme.
- Implémentez la fonction `abr_retire(abr *a, int n)`, qui retire l'élément `n` de l'arbre `a`. Si l'arbre est vide, ou si l'arbre ne contient pas `n`, c'est une erreur. Pour un arbre non-vide, il faut distinguer plusieurs cas, en fonction de la comparaison de `n` à la valeur à la racine; mais aussi suivant que les sous-arbres gauches et droits soient vides ou pas. Vous pourrez utiliser la fonction `abr_max()`.

Bien que vous pourrez trouver sur Internet le pseudo-code d'une routine de retrait, il vous est fortement recommandé de trouver vous-même les traitements à effectuer dans les différents cas.



Etape 3 : utilisation d'un arbre binaire de recherche: paradoxe des anniversaires.

Afin d'illustrer l'usage d'un arbre binaire de recherche, nous allons écrire un programme qui calcule empiriquement la probabilité du paradoxe des anniversaires, pour une population `N` donnée.

Le *paradoxe des anniversaires* est un résultat mathématique qui n'a rien de paradoxal; il choque simplement notre intuition.

Une version restreinte dit ceci: si vous demandez les dates de naissance de 25 personnes, il y a plus d'une chance sur deux que vous constatiez que 2 de ces personnes sont nées le même jour dans l'année (on ne tient pas compte de l'année, on ne compare que le jour dans le mois et le mois.)

Ecrire un programme qui définit une fonction `tentative(int N)`, qui tire `N` nombres au hasard entre 1 et 365, les range dans un arbre binaire de recherche, et vérifie ainsi si ces `N` nombres sont tous différents ou pas. Elle doit retourner 1 si deux nombres au moins sont égaux.

Utiliser cette fonction, en l'itérant 10000 fois, pour calculer la probabilité empirique qu'une collision se produise lorsque N personnes sont considérées.

Vous trouverez en ligne des "calculateurs du paradoxe des anniversaires" (birthday paradox calculator), qui vous permettront de vérifier vos résultats.

Tracer la courbe de l'évolution de cette probabilité pour N variant entre 20 et 30.



Etape 4 : tables dynamiques: prise en main.

Vous trouverez parmi les fichiers fournis les fichiers `table_dynamique.h/table_dynamique.c`, qui implémentent la structure de données *table dynamique*. Il s'agit d'une structure de type tableau, mais dont la taille croît automatiquement en fonction des besoins; dans le cas de cette version, l'augmentation de taille n'a lieu qu'à l'occasion des écritures de valeurs dans la structure. Les indices utilisés sont les entiers naturels, donc positifs ou nuls.

Notez que si la structure change de taille automatiquement, et fait donc des allocations mémoires sans intervention du code appelant, il est tout de même nécessaire de libérer la mémoire de la structure manuellement après utilisation.

- Utilisez la structure de table dynamique pour calculer à partir de quel indice les valeurs de la suite de Fibonacci dépassent un milliard. Vous initialiserez la suite avec $F_0 = 0$, $F_1 = 1$.



Etape 5 : arbres binaires complets implicites.

Dans cette étape, nous allons manipuler des arbres binaires, mais avec un encodage et une interface différents de ceux du module `arbre.h/arbre.c`.

Il s'agit d'arbres binaires implicites. Cela signifie que seules les valeurs seront explicitement encodées, la structure étant implicitement encodée par les positions relatives des valeurs les unes par rapport aux autres.

Plus concrètement, il est possible d'encoder un arbre comme un tableau, si on sait définir la position d'un élément dans le tableau à partir de sa position dans l'arbre.

C'est précisément ce que fait le module `arbre_implicite.h/arbre_implicite.c`, pour certains arbres particuliers que nous décrivons ci-dessous.

L'interface fournit des fonctions de calcul des indices: index de la racine, index du dernier éléments, index du fils gauche, respectivement fils droit d'un élément; index du parent d'un élément. Notez qu'une seule de ces fonctions a besoin de connaître l'arbre implicite manipulé pour répondre; les autres sont purement arithmétiques.

Notre intention est d'utiliser cette structure pour encoder uniquement des arbres binaires complets. Un arbre binaire est dit complet si à chaque profondeur sauf peut-être la plus grande, l'arbre a le nombre maximum possibles de noeuds à cette profondeur; et à la profondeur la plus grande, les noeuds absents sont à droite des noeuds présents dans l'arbre.

Pour ce type d'arbre, les indices utilisés dans le tableau l'encodant sont consécutifs.

L'interface fournit aussi une fonction de lecture d'un élément dont on connaît l'index; une fonction d'ajout en fin, et une fonction de retrait du dernier élément uniquement; et une fonction échangeant la place de deux éléments dans la structure.

- Ecrire le code de la fonction `abi_vers_arbre()` du module `fonctions_arbres_implicites.h`: elle doit transformer un arbre binaire complet implicite en arbre binaire au sens du module `arbre.h/arbre.c`.

- Assurez-vous d'atteindre une bonne compréhension du fonctionnement du module `arbre_implicite.h/arbre_implicite.c` via l'écriture de codes d'exemple.



Etape 6 : structure de tas.

Parmi les arbres binaires, on distingue une sous-catégorie vérifiant une propriété spécifique. On appelle *tas* un arbre vérifiant la propriété suivante: pour tout noeud de l'arbre, la valeur stockée dans le fils d'un noeud est plus grande ou égale à la valeur stockée dans le noeud lui-même. On appellera dans la suite cette propriété *propriété de tas*.

Dans cette étape, nous allons manipuler des tas encodés comme des arbres binaires complets implicites. Le module permettant de les manipuler est `tas.h/tas.c`. Les données manipulées sont de type `abi`; les fonctions du module `arbre_implicite.h/arbre_implicite.c` peuvent être utilisées directement sur un tas.

Les manipulations des tas se font classiquement en s'appuyant sur des routines internes, qui permettent de rétablir la propriété de tas lorsque celle-ci a été perturbée. Ces routines font descendre ou remonter des éléments dans le tas, par échanges successifs lorsque cela est nécessaire.

A noter que les tas sont un cas particulier de file de priorité; c'est-à-dire de structures permettant de savoir à chaque instant quel est l'élément prioritaire au sein de la structure. Dans notre cas, l'élément le plus prioritaire est l'élément le plus petit. La fonction `tas_min()` permet de connaître cet élément en temps $O(1)$.

- Parcourez le code de la fonction `tas_redescend()` dans `tas.c`. Cette routine n'est pas présente dans l'interface `tas.h`: elle n'a pas vocation à être appelée par l'utilisateur du module.
- Ecrire le code de la fonction `tas_remonte()` dans `tas.c`. Elle doit faire remonter dans le tas l'élément considéré, tant qu'il est plus petit que son parent.
- Ecrire le code de la fonction `tas ajoute()` dans `tas.c`. Cette fonction doit ajouter son argument à l'arbre implicite via `abi_ajoute()`. Comme cet ajout peut perturber la propriété de tas, elle doit ensuite appeler `tas_remonte()` sur l'élément fraîchement inséré.
- Ecrire le code de la fonction `tas_retire_min()` dans `tas.c`. Cette fonction doit lire la valeur du dernier élément de l'arbre implicite; écraser la racine de l'arbre implicite avec cette valeur (via un appel à `abi_ecrase_racine()`); et comme la propriété de tas peut être perturbée par cet écrasement, elle doit appeler `tas_redescends()` sur la racine de l'arbre implicite.



Etape 7 : tri par tas.

Une application classique des tas est le tri par tas.

Son principe est très simple. Etant donnée une liste d'objets à trier, insérer un à un les éléments dans un tas initialement vide. Puis retirer ensuite les éléments un par un du tas. Les éléments sortent du tas par ordre croissant.

- Implémenter une routine de tri par tas.
- Quelle est la complexité de cette routine de tri?